/

Course Name:	**Competitive Programming Lab**	**Semester:**	**IV**
Date of Performance:	**31 / 03 / 2026**	**DIV/ Batch No:**	**A-3**
Student Name:	**Shrived Deshmukh**	**Roll No:**	**71**

Experiment No: 5

Title: Bit Manipulation Techniques

Aim of the Experiment: To understand and Bit Manipulation Techniques for a given problem

Objective of the Experiment:
To understand and apply bit manipulation techniques for efficient data processing by performing operations such as setting, clearing, toggling, and checking specific bits using bitwise operators.

COs to be achieved:

CO 2 Analyse and optimize algorithms using amortized analysis and bit manipulation, equipping them to tackle complex computational problems.

Theory:

Bit manipulation refers to the act of directly operating on individual bits of binary data. Since computers internally represent data in binary (0s and 1s), bit manipulation allows programs to run faster, use less memory, and perform low-level operations efficiently.

Bitwise operators work at the bit level rather than on entire numbers. These operations are commonly used in embedded systems, cryptography, compression algorithms, networking, and performance-critical applications.

Common Bitwise Operators:

Operator	Name	Description
&	AND	Sets bit to 1 if both bits are 1
|	OR	Sets the bit to 1 if **at least one** of the corresponding bits is 1
^	XOR	Sets bit to 1 if bits are different
~	NOT	Inverts all bits
<<	Left Shift	Shifts bits left (multiplies by 2)
>>	Right Shift	Shifts bits right (divides by 2)

Advantages of Bit Manipulation

- Faster execution compared to arithmetic operations
- Efficient memory usage
- Useful for low-level hardware interaction
- Simplifies flag handling and permissions

Applications

- Embedded systems and microcontrollers
- Cryptography and security algorithms
- Network protocol implementation
- Data compression
- Game development and graphics

Problem Statements:

Difficulty (as per lab)	Problem	LeetCode mapping
Easy	Two unique elements (others twice)	LC 260 – Single Number III
Medium	Generate all subsets using bitmasks	LC 78 – Subsets
Difficult	Bitmask DP for assignment ($N \leq 12$)	LC 1879 – Minimum XOR Sum of Two Arrays

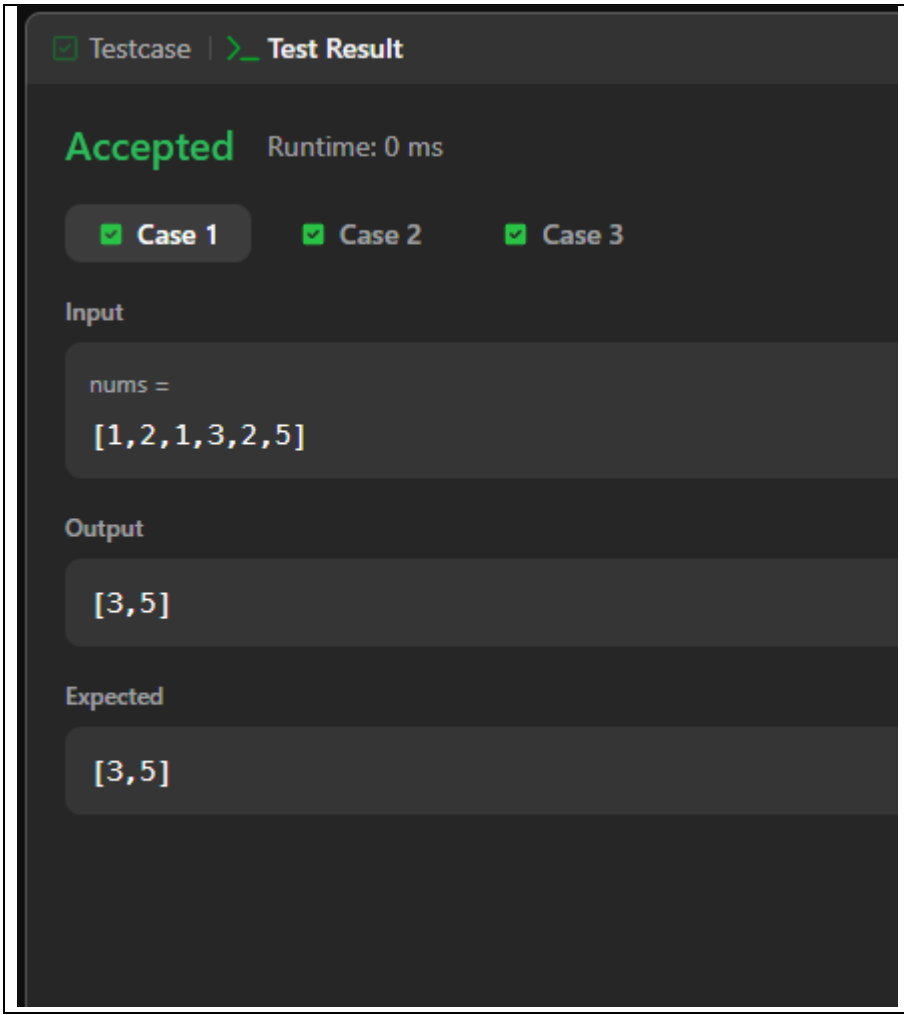
Code :

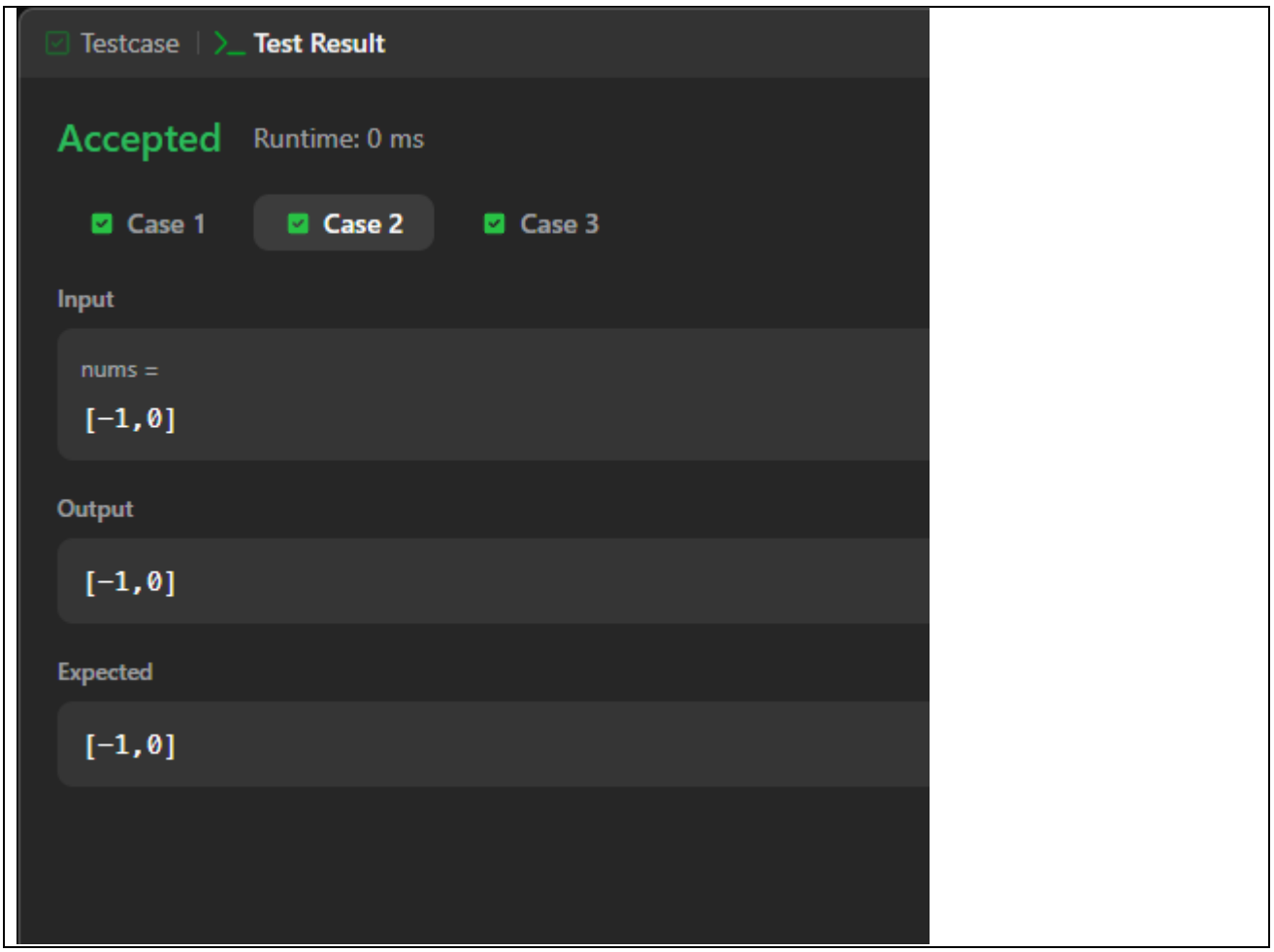
Q1) Easy :-
Code:-

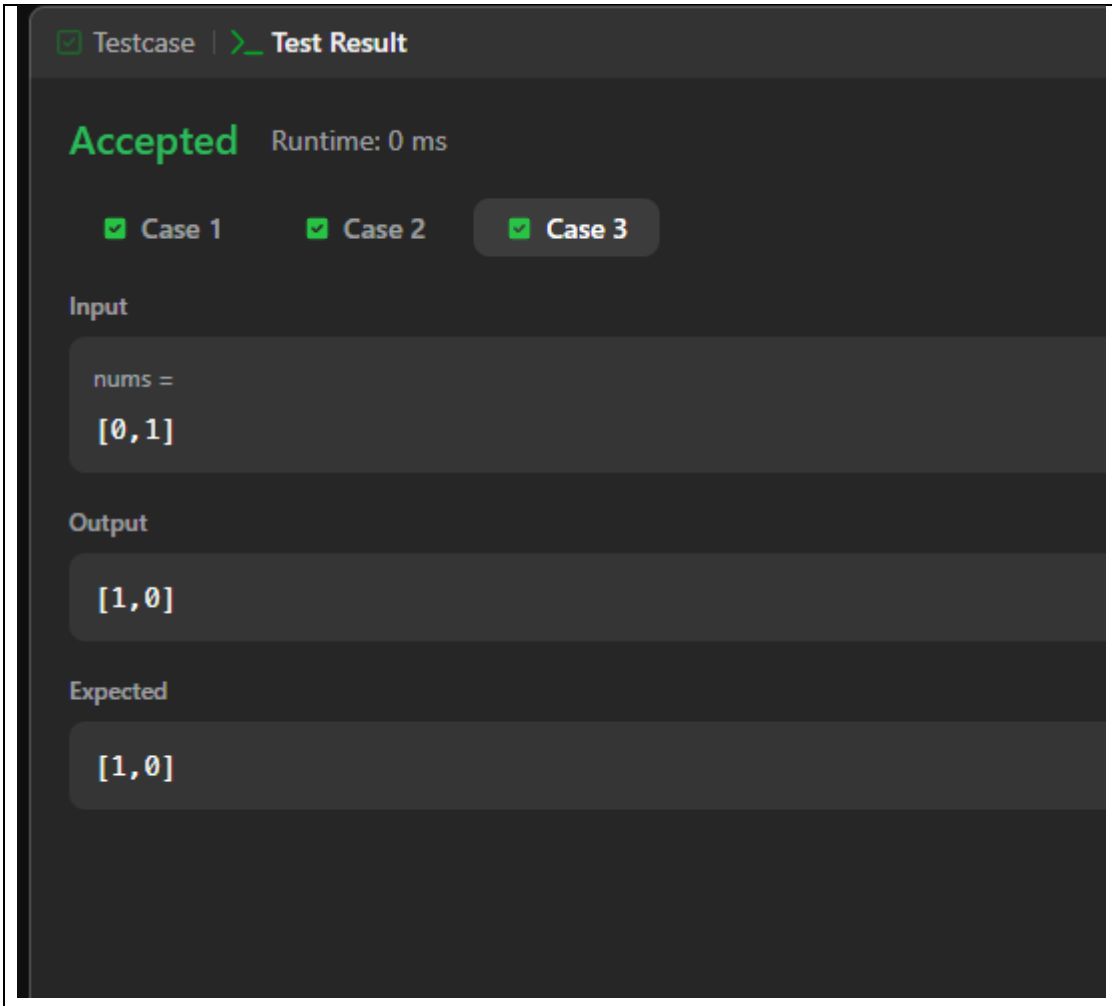
Code

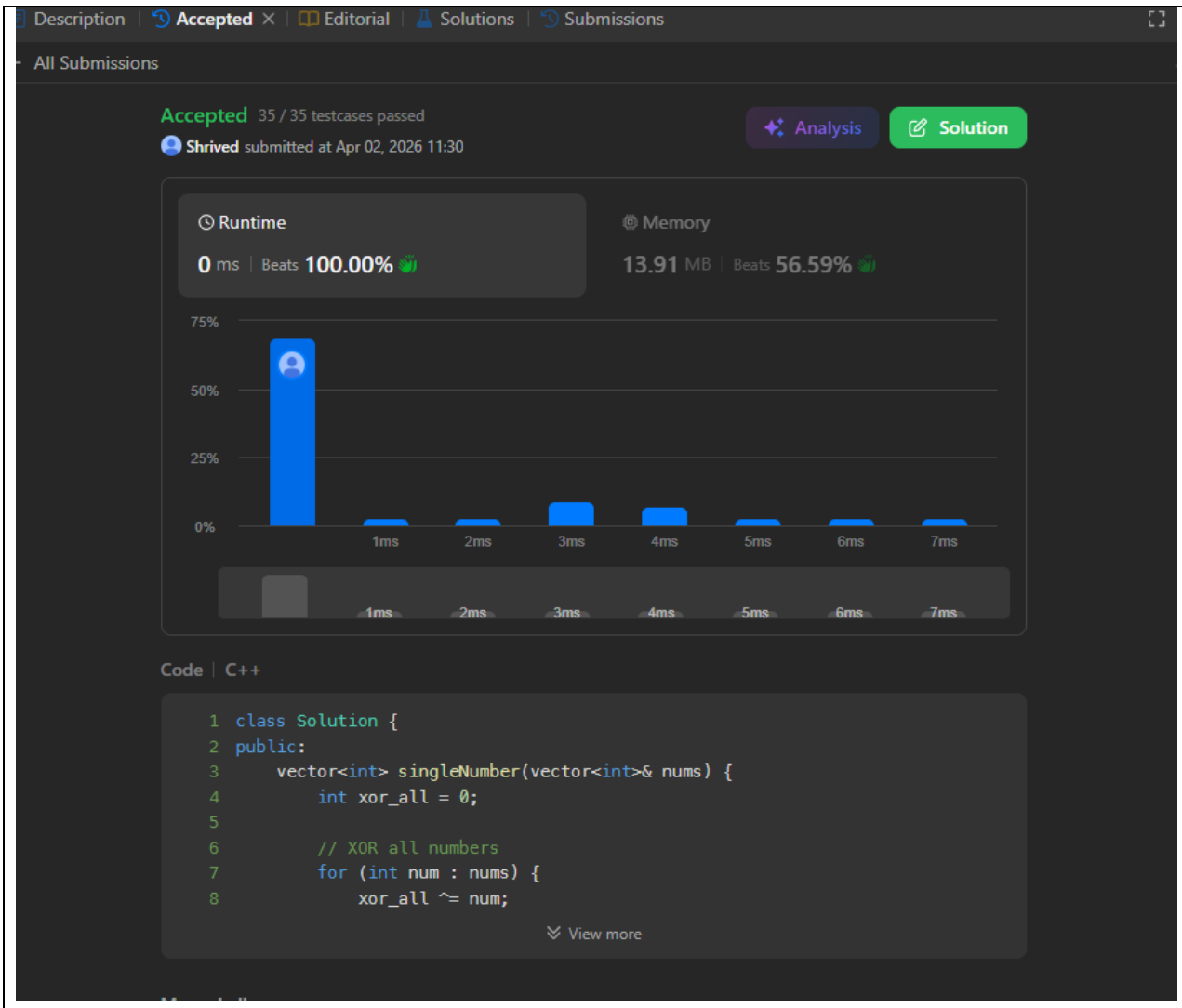
C++ Auto

```cpp
class Solution {
public:
    vector<int> singleNumber(vector<int>& nums) {
        int xor_all = 0;

        // XOR all numbers
        for (int num : nums) {
            xor_all ^= num;
        }

        // Safely get rightmost set bit
        unsigned int diff = xor_all & -((unsigned int)xor_all);

        int a = 0, b = 0;

        // Split into two groups
        for (int num : nums) {
            if (num & diff)
                a ^= num;
            else
                b ^= num;
        }

        return {a, b};
    }
};
```


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[1,2,1,3,2,5]
Output
[3,5]
Expected
[3,5]


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[-1,0]
Output
[-1,0]
Expected
[-1,0]


Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Case 3
Input
nums =
[0,1]
Output
[1,0]
Expected
[1,0]


Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 35 / 35 testcases passed
Shrived submitted at Apr 02, 2026 11:30
Analysis
Solution
Runtime
0 ms | Beats 100.00%
Memory
13.91 MB | Beats 56.59%
75%
50%
25%
0%
1ms
2ms
3ms
4ms
5ms
6ms
7ms
Code | C++
class Solution {
public:
vector<int> singleNumber(vector<int>& nums) {
int xor_all = 0;
// XOR all numbers
for (int num : nums) {
xor_all ^= num;
View more

Q2)Medium:-

Code

C++ Auto

```cpp
class Solution {
public:
    vector<vector<int>> result;

    void backtrack(int index, vector<int>& nums, vector<int>& current) {
        // Add current subset
        result.push_back(current);

        for (int i = index; i < nums.size(); i++) {
            // include nums[i]
            current.push_back(nums[i]);

            // move forward
            backtrack(i + 1, nums, current);

            // backtrack (remove last element)
            current.pop_back();
        }
    }

    vector<vector<int>> subsets(vector<int>& nums) {
        vector<int> current;
        backtrack(0, nums, current);
        return result;
    }
};
```

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[1,2,3]

Output

[[],[1],[1,2],[1,2,3],[1,3],[2],[2,3],[3]]

Expected

[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =

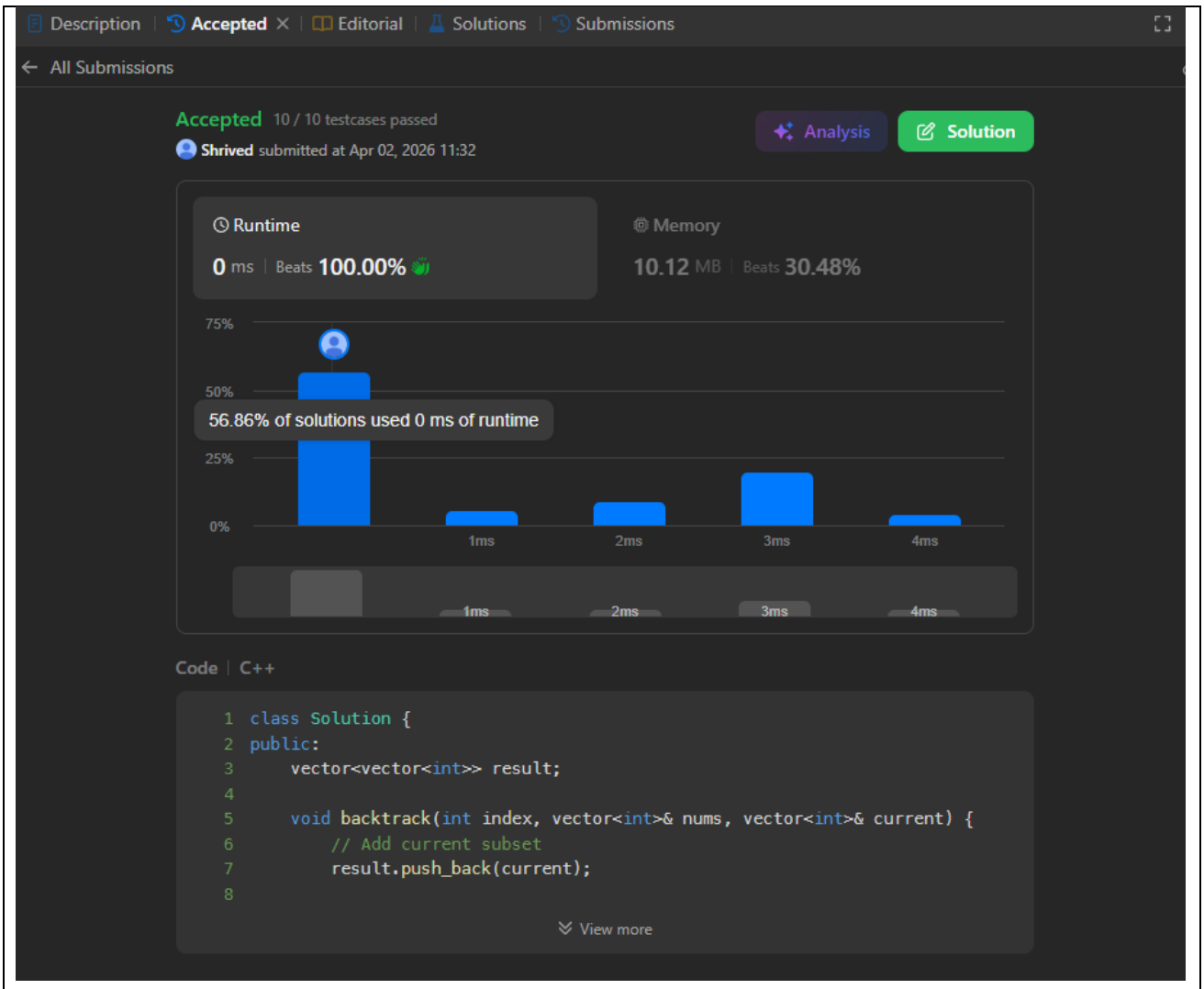
[0]

Output

[[],[0]]

Expected

[[],[0]]


Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 10 / 10 testcases passed
Shrived submitted at Apr 02, 2026 11:32
Analysis
Solution
Runtime
0 ms | Beats 100.00%
Memory
10.12 MB | Beats 30.48%
75%
50%
56.86% of solutions used 0 ms of runtime
25%
0%
1ms
2ms
3ms
4ms
Code | C++
class Solution {
public:
vector<vector<int>> result;
void backtrack(int index, vector<int>& nums, vector<int>& current) {
// Add current subset
result.push_back(current);
View more

Q3)Hard:-

Code

C++ Auto

```cpp
class Solution {
public:
    int dp[1 << 14];

    int solve(vector<int>& nums1, vector<int>& nums2, int mask, int i) {
        if (i == nums1.size()) return 0;

        if (dp[mask] != -1) return dp[mask];

        int ans = INT_MAX;

        for (int j = 0; j < nums2.size(); j++) {
            if (!(mask & (1 << j))) {
                ans = min(ans,
                          (nums1[i] ^ nums2[j]) +
                          solve(nums1, nums2, mask | (1 << j), i + 1));
            }
        }

        return dp[mask] = ans;
    }

    int minimumXORSum(vector<int>& nums1, vector<int>& nums2) {
        memset(dp, -1, sizeof(dp));
        return solve(nums1, nums2, 0, 0);
    }
};
```

Testcase | Test Result

Case 1 Case 2 +

nums1 =

[1,2]

nums2 =

[2,3]

Testcase | Test Result

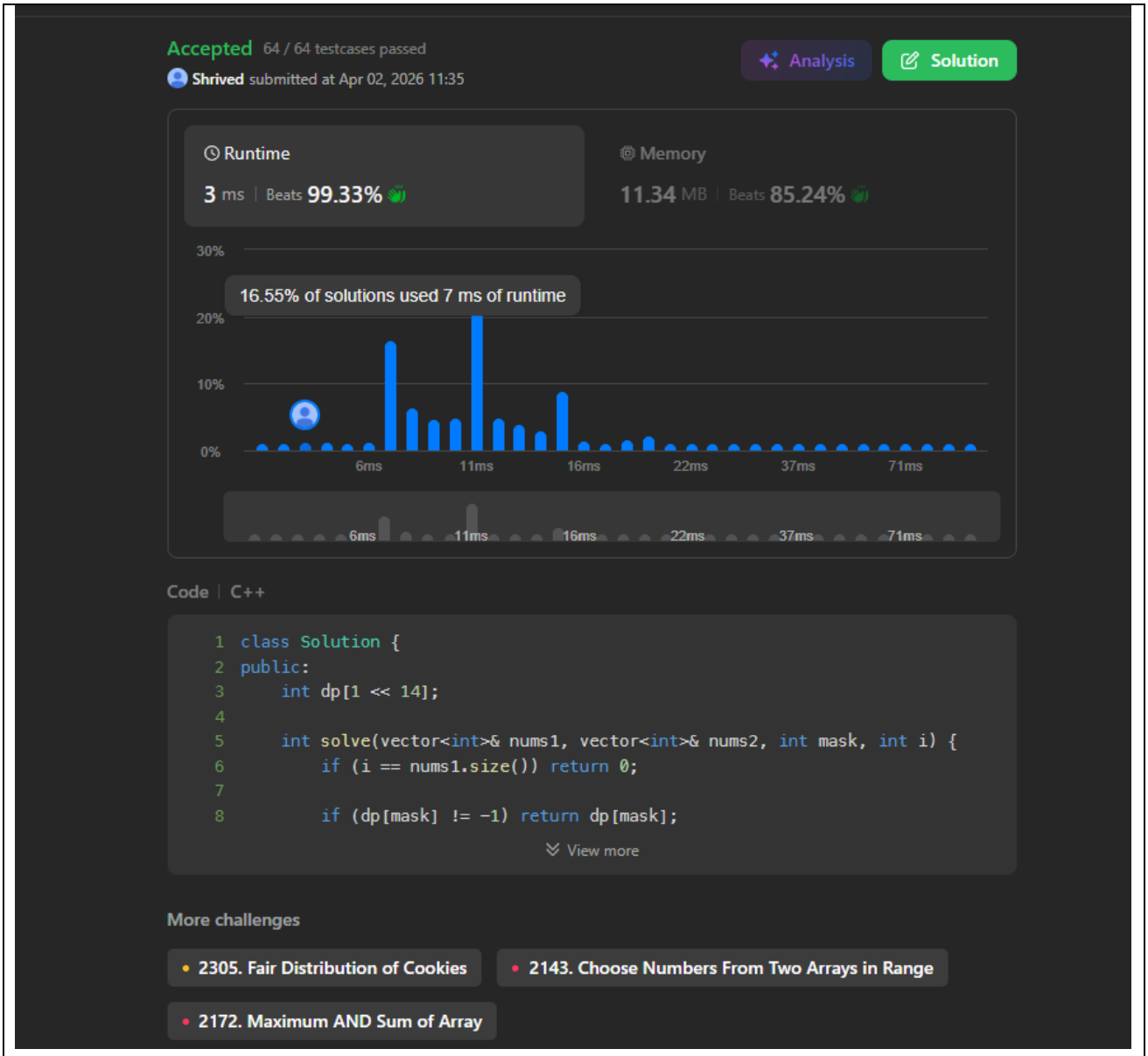
Case 1 Case 2 +

nums1 =

[1,0,3]

nums2 =

[5,3,4]



Post Lab Subjective/Objective type Questions:

1. Why are bit manipulation techniques considered faster than arithmetic operations?

Bit manipulation techniques are considered faster than arithmetic operations because they operate directly at the binary level, which is how data is represented in a computer. Bitwise operations like

AND, OR, XOR, and shifts are executed in a single CPU cycle, making them extremely efficient. In contrast, arithmetic operations such as multiplication, division, and modulo require more complex hardware logic and multiple steps to compute. Bitwise operations also avoid carry propagation, which is necessary in addition and subtraction, further improving speed. They work independently on each bit, allowing simpler and quicker execution. Additionally, operations like checking even or odd numbers (`x & 1`) or multiplying/dividing by 2 (`x << 1`, `x >> 1`) can be done much faster than their arithmetic equivalents. Although modern compilers often optimize arithmetic into bitwise instructions, understanding bit manipulation still helps in writing highly efficient code. This is why bitwise techniques are widely used in performance-critical applications like competitive programming and system-level programming.

2. Explain the difference between left shift (**<<**) and right shift (**>>**) operators with an example.

The left shift (<<) and right shift (>>) operators are used to move the bits of an integer either to the left or right. The left shift operator (<<) moves all bits to the left by a specified number of positions, filling the vacant least significant bits with 0, and effectively multiplying the number by 2 for each shift. For example, shifting `5` (`0101` in binary) left by 2 positions gives `20` (`010100` in binary). In contrast, the right shift operator (>>) moves all bits to the right by a specified number of positions, and the vacant most significant bits are filled with 0 for unsigned numbers or the sign bit for signed numbers, effectively dividing the number by 2 for each shift. For instance, shifting `20` (`10100` in binary) right by 2 positions results in `5` (`000101` in binary). In summary, << multiplies a number by powers of 2 by shifting bits left, while >> divides a number by powers of 2 by shifting bits right.

Conclusion:

Bit manipulation operators like left shift (<<) and right shift (>>) provide a fast and efficient way to perform multiplication and division by powers of two at the binary level. Understanding how they move bits and affect numbers allows programmers to write optimized code for arithmetic operations, low-level programming, and performance-critical applications. They are fundamental tools in computer science for manipulating data directly at the bit level.

Signature of faculty in-charge with Date: